

Build Task — Project 4 as an API — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper FastAPI/SQLite/Docker, split into the suggested files). Read Basic first even if you've done every Practice Exercise above — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're combining everything from the Practice Exercises above into one working service: a FastAPI route that runs Project 4's multi-agent task, a SQLite table that remembers every run, log lines tagged with which request they belong to, a Dockerfile that packages the whole thing, and a simple limit on how often one caller can hit the route.

Almost nothing here is new — it's `chat_route` (a route wrapping real logic, 400/500, `logger.exception`, request IDs) plus `sqlite_persistence` (saving runs, `GET /runs/{id}` with a 404) plus `validation_422` (`INSERT` → `lastrowid` → `UPDATE`) plus `db_failure_and_docker` (Dockerfile, `.dockerignore`, startup env check). The one new piece is rate limiting, and it's built from a plain dict, a list, and `time.time()` — nothing you haven't used.

The exact pieces:

- `POST /run-task` — a route wrapping Project 4's task-running function, same shape as `chat_route`'s `/chat`.
- A `runs` table with `id`, `input`, `output`, `log`, `status`, `created_at` — `sqlite_persistence`'s table, plus `log` and `status` columns.
- `GET /runs/{id}` — look up one run by ID, same as `sqlite_persistence`'s Approach 3.
- `GET /health` — same as `health_route`.
- A request ID, made once per request in a middleware and put on every log line — same as `chat_route`'s Approach 4.
- A dictionary of recent request times per caller, to reject a caller over the limit with a `429`.

Intermediate Version

Think of the Build Task as 3 layers, each one you've already built separately: the **API layer** (`chat_route` — typed request in, typed response out, 4xx/5xx handled correctly, real errors logged on the server only), the **storage layer** (`sqlite_persistence` + `validation_422` — but now with a `status` that changes as the run moves: `"started"` when the row is first written, `"completed"` or `"failed"` once it's done), and the **operations layer** (`db_failure_and_docker` — Dockerfile, secrets only via env vars, startup check — plus rate limiting).

Why status needs 3 states, not just "done": a task run can fail partway through — an agent might crash, an API call might time out. Saving a `"started"` row *before* running the task (in its own `with conn:`), then updating it to `"completed"` or `"failed"` after, means a crashed run still leaves a

real, honest record. That's the one deliberate difference from `validation_422`'s Approach 3, which rolled both writes back together.

Why the request ID matters here specifically: a multi-agent run writes many log lines per request. Without a shared ID on each, you can't tell which lines belong to which request once two people call your API at the same time.

The honest limitation of an in-memory rate limiter (the Requirements ask you to write it down): a Python dict forgets everything when the process restarts, and if you run more than one copy of the API (which real deployments do), each copy has its *own* dict — so a caller could get 3x their limit by landing on 3 copies. The real fix is a shared store all copies can see (Redis is the standard choice). Put this in a comment right next to the dict.

Keeping secrets out of logs: never log the API key, request headers, or `str(exc)` from a provider SDK error without looking at it first. `logger.exception()` with your own short message, as in `chat_route`, is the safe default.

The exact pieces:

- `api/schemas.py` → `TaskRequest(task: str)`, `TaskResponse(run_id, result, status)` — like `chat_schemas.py`.
- `db/database.py` → `get_connection()`, `insert_started_run()`, `finish_run()`, `fail_run()`, `fetch_run()` — like `runs_db.py`, each write in its own `with conn:`.
- `db/models.py` → a `Run(BaseModel)` for `GET /runs/{id}`'s `response_model`; `output` and `log` are `str | None`, since they're empty while a run is "started" or "failed".
- `api/routes.py` → an `APIRouter` with the 3 routes, plus `check_rate_limit(caller_key)`, called with `http_request.client.host` (the caller's IP address).
- `api/main.py` → the startup env check, `app.include_router(router)`, and the request-ID middleware.
- `"\n".join(steps)` — turns the agent's list of steps into one text value for the `log` column.
- `test_api.py` → `TestClient`, one check per row of the Test Cases table, printed with `# expected:` comments — like `health_route`'s Approach 2.

Difference between Basic and Intermediate: Basic names the pieces and maps each one straight back to a Practice Exercise. Intermediate explains why a 3-state status matters once a run can fail partway through, why the request ID matters under real traffic, the honest limit of an in-memory rate limiter, and how the pieces split across the suggested files.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
one file, main.py:
    connect to runs.db, create table runs
        (id, input, output, log, status, created_at) if missing

    route GET /health: return {"status": "ok"}

    route POST /run-task, takes a TaskRequest:
        with conn: insert a row with status "started", keep lastrowid
        try:
            output, steps = run project 4's task
        except:
            with conn: update that row to status "failed"
            raise a 500 with a plain message
        with conn: update that row: output, log, status "completed"
        return run_id, result, status

    route GET /runs/{id}:
        look up the row; if missing: 404; return it

    Dockerfile: same shape as db_failure_and_docker's
```

Here's the `POST /run-task` route — fill in the two `UPDATE` s yourself:

```
@app.post("/run-task")
def run_task(request: TaskRequest):
    now = datetime.now(timezone.utc).isoformat()
    with conn:
        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
            (request.task, "started", now),
        )
    run_id = cursor.lastrowid
    try:
        output, steps = run_project_4_task(request.task)
    except Exception:
        # your turn: with conn, UPDATE this run's status to "failed"
```

```

...
    raise HTTPException(status_code=500, detail="Internal error.")
log_text = "\n".join(steps)
# your turn: with conn, UPDATE output, log, and status "completed"
...
return {"run_id": run_id, "result": output, "status": "completed"}

```

Intermediate Version

split into the suggested files:

- api/main.py, api/routes.py, api/schemas.py
- db/database.py, db/models.py
- logging_setup.py (copied from Doc01, unchanged)

db/database.py: get_connection, insert_started_run, finish_run,
fail_run, fetch_run – each write inside its own "with conn:"

api/main.py: startup env check (MissingConfigError), app,
include_router, request-ID middleware

api/routes.py:

- rate limiting: a dict of {caller: [timestamp, timestamp, ...]}
- on each request: keep only timestamps inside the window, then:
 - if len(recent) >= limit: raise 429
 - else: add this request's timestamp, carry on
- every log line starts with "[request_id]"

Here's the rate limiter — fill in the loop yourself:

```

import time
from fastapi import HTTPException

RATE_LIMIT = 5          # max requests per caller...
RATE_WINDOW_SECONDS = 60 # ...inside this many seconds
# LIMITATION: write the honest limitation here, in your own words
request_times = {}      # caller -> list of request timestamps

def check_rate_limit(caller_key: str) -> None:
    now = time.time()
    old_times = request_times.get(caller_key, [])
    recent = []
    # your turn: loop over old_times, append the ones where
    # now - t < RATE_WINDOW_SECONDS to recent
    ...
    if len(recent) >= RATE_LIMIT:

```

```
raise HTTPException(status_code=429, detail="Rate limit exceeded")
recent.append(now)
request_times[caller_key] = recent
```

Expected output for the 6th request within 60 seconds from the same caller:

```
429 {"detail":"Rate limit exceeded"}
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

`run_project_4_task()` below stands in for Project 4's real multi-agent entry point — it returns the result plus a list of agent steps, and crashes on purpose for the input `"boom"`. Everything around it (routing, storage, logging, rate limiting, Docker) is what this Build Task is about, and doesn't change once you swap the real function in. Read both depths — they're not "wrong, right," they're 2 real, valid ways to meet the same Requirements, with real tradeoffs between them.

Basic Version

Approach 1 — one file, synchronous

Story: before splitting anything into folders, get the whole loop — request in, row saved, task run, row updated, reply out — working in one file you can read top to bottom. **If not:** a bug could be in any of five files at once, and you'd be debugging the file layout instead of the logic.

```
# practice/build_task/main.py
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        log TEXT,
        status TEXT,
        created_at TEXT
    )
""")

class TaskRequest(BaseModel):
    task: str

def run_project_4_task(task_input: str):
```

```

# stand-in for Project 4's real graph – swap in the real one
if task_input == "boom":
    raise RuntimeError("simulated internal failure")
steps = [
    "supervisor -> research_agent",
    "supervisor -> writer_agent",
    "supervisor -> reviewer_agent: approved",
]
return "result for: " + task_input, steps

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/run-task")
def run_task(request: TaskRequest):
    now = datetime.now(timezone.utc).isoformat()
    with conn:
        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
            (request.task, "started", now),
        )
    run_id = cursor.lastrowid
    try:
        output, steps = run_project_4_task(request.task)
    except Exception:
        with conn:
            conn.execute(
                "UPDATE runs SET status = ? WHERE id = ?",
                ("failed", run_id),
            )
        raise HTTPException(status_code=500, detail="Internal error.")
    log_text = "\n".join(steps)
    with conn:
        conn.execute(
            "UPDATE runs SET output = ?, log = ?, status = ? WHERE id = ?",
            (output, log_text, "completed", run_id),
        )
    return {"run_id": run_id, "result": output, "status": "completed"}

@app.get("/runs/{run_id}")
def get_run(run_id: int):
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="run not found")
    return dict(row)

```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Expected output (shown wrapped just to fit the page — each reply is really one line):

```
$ curl -X POST localhost:8000/run-task \
  -H "Content-Type: application/json" -d '{"task": "summarize this"}'
{"run_id":1,"result":"result for: summarize this","status":"completed"}
$ curl localhost:8000/runs/1
{"id":1,"input":"summarize this","output":"result for: summarize this",
  "log":"supervisor -> research_agent\nsupervisor -> writer_agent\n...",
  "status":"completed","created_at":"2026-01-01T00:00:00.000000+00:00"}
```

This covers the route, the saved run with its log, the 422 (for free), the 500, and the 404. It's still missing the per-request log tags, the rate limit, the startup env check, and the suggested file split — all below.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — split into the suggested files, request-ID logging, and a rate limiter

Copy Doc01's `logging_setup.py` into `practice/build_task/` first, unchanged.

Story — `db/database.py` : every SQL statement lives here, behind small named functions, like `runs_db.py` in `sqlite_persistence`. Each write gets its own `with conn:`, so the "started" row is saved on its own and a crash later leaves an honest "failed" record. **If not:** SQL would be scattered through the routes, and a crashed run could leave either nothing or a row stuck at "started" forever.

```
# practice/build_task/db/database.py
import sqlite3
from datetime import datetime, timezone

# why: the Build Task's one table – same columns as sqlite_persistence,
# plus log and status
CREATE_RUNS_TABLE = """
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        log TEXT,
        status TEXT,
        created_at TEXT
    )
"""

def get_connection():
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute(CREATE_RUNS_TABLE)
    return conn

def insert_started_run(conn, task_input):
    now = datetime.now(timezone.utc).isoformat()
    # why: its own transaction – the "started" row is saved right away,
    # so a crash later still leaves a record to look up
    with conn:
        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
```

```

        (task_input, "started", now),
    )
    return cursor.lastrowid

def finish_run(conn, run_id, output, log_text):
    with conn:
        conn.execute(
            "UPDATE runs SET output = ?, log = ?, status = ? WHERE id = ?",
            (output, log_text, "completed", run_id),
        )

def fail_run(conn, run_id):
    with conn:
        conn.execute(
            "UPDATE runs SET status = ? WHERE id = ?",
            ("failed", run_id),
        )

def fetch_run(conn, run_id):
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    # when: no row with this id – the route turns None into a 404
    if row is None:
        return None
    return dict(row)

```

Story — `db/models.py` and `api/schemas.py` : the shapes going in and out of the API, written down once, like `RunRecord` and `chat_schemas.py`. **If not:** `GET /runs/{id}` could leak any column you add to the table later, and a wrong field name would only be found by the client.

```

# practice/build_task/db/models.py
from pydantic import BaseModel

# why: one saved run's shape, checked on the way out –
# same idea as RunRecord in sqlite_persistence
class Run(BaseModel):
    id: int
    input: str
    output: str | None    # None while the run is "started" or "failed"
    log: str | None
    status: str
    created_at: str

```

```

# practice/build_task/api/schemas.py
from pydantic import BaseModel

```

```

class TaskRequest(BaseModel):
    task: str

class TaskResponse(BaseModel):
    run_id: int
    result: str
    status: str

```

Story — `api/routes.py` : the three routes on an `APIRouter` (like `health_routes.py`), plus the rate limiter. `POST /run-task` is `chat_route`'s Approach 4 with a database around it: the same 400/500 split, the same `logger.exception`, the same `[request_id]` at the start of every log line. **If not:** a failed run would leave no trace in the logs, and one caller could run up your OpenAI bill with no limit at all.

```

# practice/build_task/api/routes.py
import time
from fastapi import APIRouter, HTTPException, Request

from api.schemas import TaskRequest, TaskResponse
from db.database import get_connection, insert_started_run
from db.database import finish_run, fail_run, fetch_run
from db.models import Run
from logging_setup import get_logger    # Doc01 Build Task

logger = get_logger(__name__)
router = APIRouter()
conn = get_connection()

RATE_LIMIT = 5            # max requests per caller...
RATE_WINDOW_SECONDS = 60  # ...inside this many seconds
# LIMITATION: in-memory only. It resets on every restart, and each
# worker/container keeps its own separate counts – so 3 containers
# really allow 3 x RATE_LIMIT. A shared store (like Redis) fixes that.
request_times = {}        # caller -> list of request timestamps

def check_rate_limit(caller_key: str) -> None:
    now = time.time()
    old_times = request_times.get(caller_key, [])
    recent = []
    for t in old_times:
        # how: keep only the timestamps still inside the window
        if now - t < RATE_WINDOW_SECONDS:
            recent.append(t)
    if len(recent) >= RATE_LIMIT:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")

```

```

recent.append(now)
request_times[caller_key] = recent

def run_project_4_task(task_input: str):
    # stand-in for Project 4's real graph – swap in the real one
    # why: a fake crash on command, same trick as chat_route's "boom"
    if task_input == "boom":
        raise RuntimeError("simulated internal failure")
    steps = [
        "supervisor -> research_agent",
        "supervisor -> writer_agent",
        "supervisor -> reviewer_agent: approved",
    ]
    return "result for: " + task_input, steps

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest, http_request: Request) -> TaskResponse:
    request_id = http_request.state.request_id
    check_rate_limit(http_request.client.host)
    # why: "" passes Pydantic (it IS a str) – our own 400, like chat_route
    if not request.task.strip():
        raise HTTPException(status_code=400, detail="task cannot be empty")

    run_id = insert_started_run(conn, request.task)
    logger.info("[%s] run %s started", request_id, run_id)
    try:
        output, steps = run_project_4_task(request.task)
    except Exception:
        fail_run(conn, run_id)
        # how: full traceback in the server log only
        logger.exception("[%s] run %s failed", request_id, run_id)
        raise HTTPException(
            status_code=500, detail="Internal error. Please try again later."
        )

    log_text = "\n".join(steps)
    finish_run(conn, run_id, output, log_text)
    logger.info("[%s] run %s completed", request_id, run_id)
    return TaskResponse(run_id=run_id, result=output, status="completed")

@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id: int):
    run = fetch_run(conn, run_id)

```

```

if run is None:
    raise HTTPException(status_code=404, detail="run not found")
return run

```

Story — `api/main.py`: the one file uvicorn starts. It checks the required setting first, then builds the app, adds the routes, and adds the request-ID middleware from `chat_route`'s Approach 4. **If not:** a container started without its key would look healthy and fail on every real request, and log lines from two requests would be impossible to tell apart.

```

# practice/build_task/api/main.py
import os
import uuid
from fastapi import FastAPI, Request

from api.routes import router

class MissingConfigError(Exception):
    """Same custom error as Doc01's exceptions.py."""

# why: fail at startup, not on the first real request
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY:
    raise MissingConfigError(
        "OPENAI_API_KEY is not set – pass it with docker run -e"
    )

app = FastAPI()
app.include_router(router)

# why: one short ID per request, made in one place for every route
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response

```

Expected output (`OPENAI_API_KEY=sk-test-123 uvicorn api.main:app`, then the same `curl` as Basic):

```

HTTP/1.1 200 OK
x-request-id: a1b2c3d4

```

```
{"run_id":1,"result":"result for: summarize this","status":"completed"}
```

Server log for that request (both lines share the same ID):

```
2026-01-01 00:00:00,000 api.routes INFO [a1b2c3d4] run 1 started
2026-01-01 00:00:00,002 api.routes INFO [a1b2c3d4] run 1 completed
```

Approach 2 — Dockerfile, .dockerignore, and test_api.py

Story: Approach 1 runs on your laptop; this makes it run anywhere, and proves every row of the Test Cases table in one command. The Dockerfile and .dockerignore are db_failure_and_docker's, pointed at api.main:app. **If not:** "it works" would mean "it worked once, on my machine, when I clicked around" — not something you could hand to anyone.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# practice/build_task/.dockerignore
.env
.git
__pycache__/
*.pyc
.venv/
*.db
```

```
# practice/build_task/test_api.py
# Run from inside practice/build_task/:
# OPENAI_API_KEY=sk-test-123 python test_api.py
from fastapi.testclient import TestClient

from api.main import app
from api.routes import conn

client = TestClient(app)

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]
```

```

# 1. Valid POST /run-task: 200, result returned, row saved
response = client.post("/run-task", json={"task": "summarize this"})
print(response.status_code)                # expected: 200
run_id = response.json()["run_id"]
saved = client.get("/runs/" + str(run_id)).json()
print(saved["status"])                     # expected: completed

# 2. Bad request body: 422, no database write
before = row_count()
response = client.post("/run-task", json={})
print(response.status_code)                # expected: 422
print(row_count() == before)               # expected: True

# 3. Fake internal error: 500, plain message, full trace in logs only
response = client.post("/run-task", json={"task": "boom"})
print(response.status_code)                # expected: 500
print(response.json())
# expected: {'detail': 'Internal error. Please try again later.'}

# 4. Wrong run ID: 404
print(client.get("/runs/999999").status_code) # expected: 404

# 5. Over the rate limit: 429
# (2 of the 5 allowed requests were used above: tests 1 and 3 –
# the 422 request never reached the route, so it didn't count)
statuses = []
for i in range(4):
    response = client.post("/run-task", json={"task": "x"})
    statuses.append(response.status_code)
print(statuses)                           # expected: [200, 200, 200, 429]

```

Expected output (`OPENAI_API_KEY=sk-test-123 python test_api.py` ; the server's own log lines, including test 3's full traceback, are printed too and left out here):

```

200
completed
422
True
500
{'detail': 'Internal error. Please try again later.'}
404
[200, 200, 200, 429]

```

Expected output for the container:

```

$ docker build -t project-5-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 project-5-api
$ curl localhost:8000/health
{"status":"ok"}
$ docker history --no-trunc project-5-api | grep "sk-test-123"
(no output)
$ docker run project-5-api
...
api.main.MissingConfigError: OPENAI_API_KEY is not set – pass it with
docker run -e

```

(That last error is really one line — shown wrapped to fit the page.)

Difference from Basic: Approach 1 splits the single file into the suggested `api/` and `db/` layout, adds per-request log tagging via middleware, a 400 for an empty task, and a working — honestly limited — rate limiter, with its limitation written into the code. Approach 2 doesn't touch route logic — it closes out the operational requirements: a Dockerfile and `.dockerignore` that keep secrets out of every image layer, a startup check that stops the container loudly, and a `test_api.py` covering every row of the Test Cases table.

Which one should you actually write? Basic Approach 1 first — get the whole loop working in one file before splitting it. Then Intermediate Approach 1 and 2 together: that's what the Requirements ask for, and what you'd actually ship. The route stays synchronous here on purpose (a plain `def` route runs in FastAPI's thread pool, so other requests keep moving). Once the real Project 4 pipeline takes long enough that clients time out waiting, the next step is the "queues for long-running agent work" item under **Later** in this document's Topic tiers — not something this Build Task needs.